Các thuật toán duyệt đồ thị

NHÓM 3

Ngày 22 tháng 3 năm 2026

Nội dung

1. DFS
2. Ứng dụng và tính chất DFS
3. BFS
4. Ứng dụng và tính chất BFS

DFS là gì?

DFS (Depth First Search) là thuật toán duyệt đồ thị theo chiều sâu.

Ví dụ trực quan (mạng bạn bè):
Giả sử bạn bắt đầu từ bạn A

- Bạn chọn 1 người bạn của A để đi tiếp
- Sau đó tiếp tục đi sâu vào bạn của người đó
- Tiếp tục cho đến khi không còn ai mới
- Khi bí $\rightarrow$ quay lại và thử nhánh khác

Cách DFS hoạt động:

- Luôn đi theo một nhánh sâu nhất
- Khi không đi tiếp được $\rightarrow$ quay lui (backtrack)
- Sau đó thử các nhánh khác

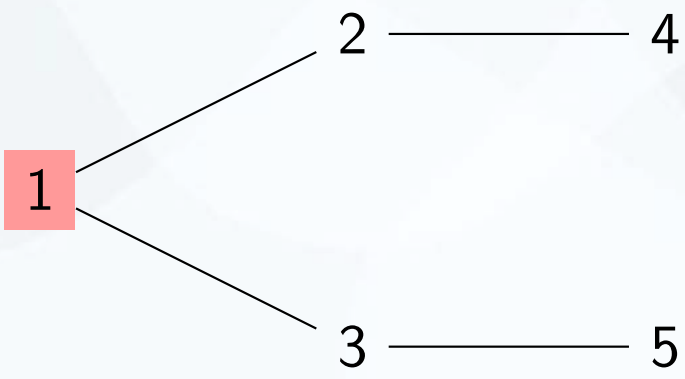
DFS - Code và giải thích

```
vector<int> adj[1000];
bool visited[1000];

void dfs(int u){
    visited[u] = true;

    for(int v : adj[u]){
        if(!visited[v]){
            dfs(v);
        }
    }
}
```

Giải thích

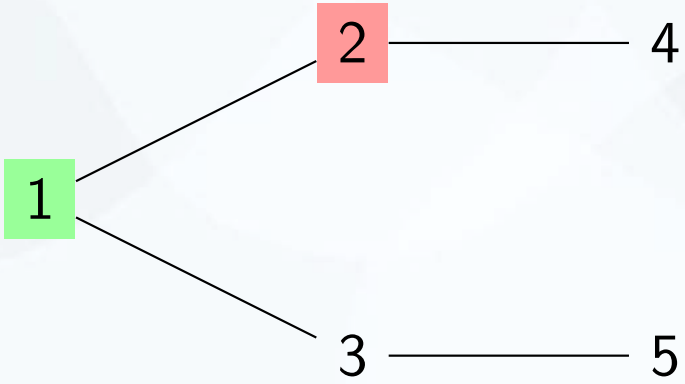
- adj: danh sách kề
- visited: đánh dấu
- dfs(u): bắt đầu tại u
- visited[u]: đã thăm
- for: duyệt kề
- if: tránh lặp
- dfs(v): đi sâu

DFS bước 1



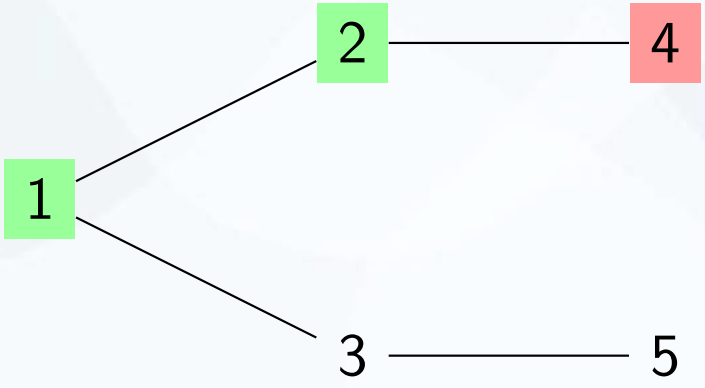
Bắt đầu từ đỉnh 1

DFS bước 2



Đi từ $1 \rightarrow 2$

DFS bước 3



Đi sâu: $1 \rightarrow 2 \rightarrow 4$

Ứng dụng DFS (giải thích chi tiết)

- Tìm thành phần liên thông Duyệt từ một đỉnh, tất cả các đỉnh đi tới được sẽ thuộc cùng một thành phần. Lặp lại với các đỉnh chưa thăm để đếm số thành phần.
- Phát hiện chu trình Nếu trong DFS gặp lại một đỉnh đã thăm nhưng không phải cha $\rightarrow$ tồn tại chu trình.
- Giải mê cung DFS thử từng đường đi $\rightarrow$ nếu sai thì quay lui $\rightarrow$ tìm được đường tới đích.
- Backtracking DFS là nền tảng cho các bài toán thử tất cả khả năng như: sinh hoán vị, tổ hợp,...

Tính chất của DFS

- Duyệt theo chiều sâu Luôn đi sâu vào một nhánh cho đến khi không còn đỉnh chưa thăm.
- Có cơ chế quay lui (Backtracking) Khi gặp ngõ cụt $\rightarrow$ quay lại đỉnh trước đó để tiếp tục duyệt nhánh khác.
- Tạo ra cây DFS Trong quá trình duyệt, các cạnh được chọn sẽ tạo thành một cây (hoặc rừng DFS).
- Phân loại cạnh Trong DFS có thể chia cạnh thành:
  - Tree edge (cạnh cây)
  - Back edge (cạnh ngược $\rightarrow$ phát hiện chu trình)
  - Forward edge
  - Cross edge
- Độ phức tạp Thời gian chạy: $O(V + E)$

BFS là gì?

BFS (Breadth First Search) là thuật toán duyệt đồ thị theo chiều rộng.

Ví dụ trực quan (mạng bạn bè):
Giả sử bạn bắt đầu từ bạn A

- Level 0: A (bạn)
- Level 1: bạn trực tiếp của A
- Level 2: bạn của bạn A
- Level 3: bạn của level 2

Cách BFS hoạt động:

- Duyệt hết bạn gần nhất trước
- Sau đó mới tới bạn xa hơn
- Mỗi lần lan ra giống như sóng lan

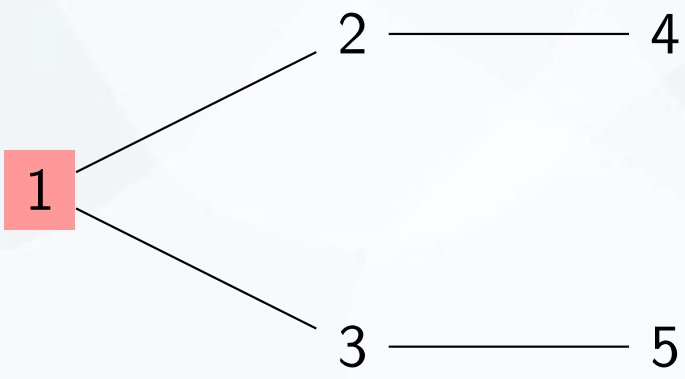
BFS - Code và giải thích

```
void bfs(int s){
    queue<int> q;

    q.push(s);
    visited[s] = true;

    while(!q.empty()){
        int u = q.front();
        q.pop();

        for(int v : adj[u]){
            if(!visited[v]){
                visited[v] =
                    true;
                q.push(v);
            }
        }
    }
}
```

Giải thích

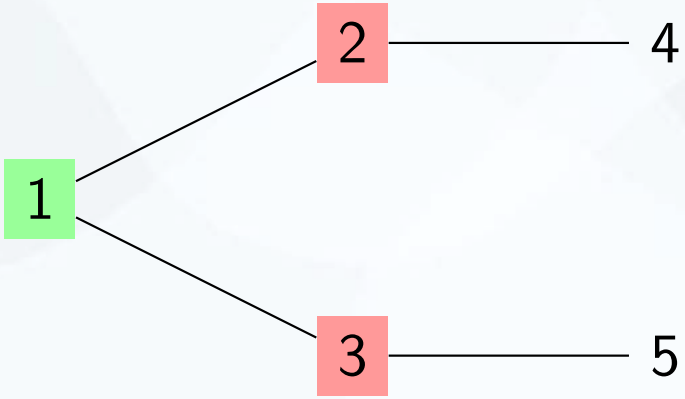
- queue: theo lớp
- push: thêm
- pop: lấy ra
- visited: tránh lặp

BFS bước 1



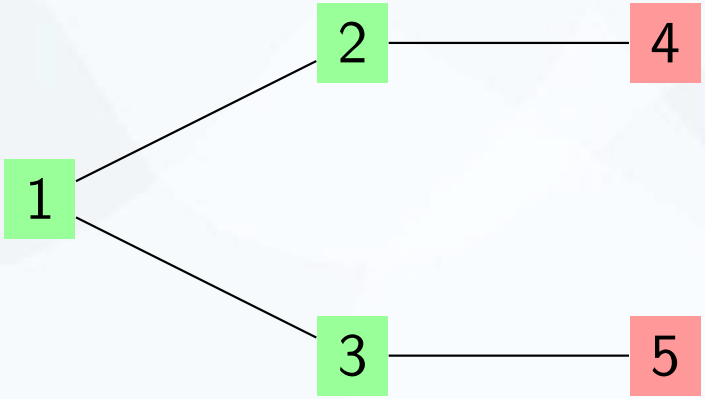
Bắt đầu từ đỉnh 1

BFS bước 2



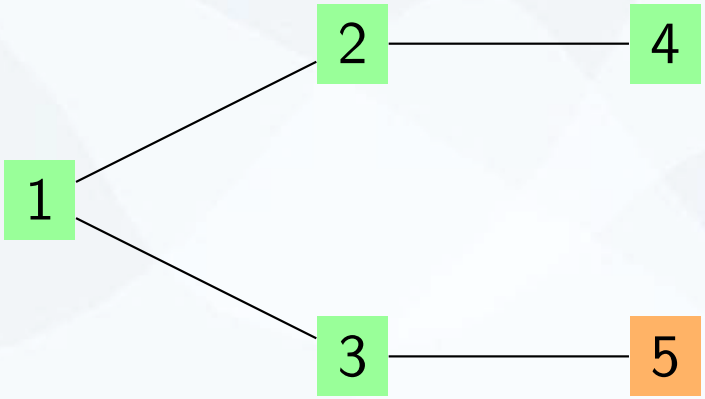
Từ 1 $\rightarrow$ duyệt tất cả đỉnh kề (2, 3)

BFS bước 3



Tiếp tục duyệt level tiếp theo (4, 5)

Kết quả BFS



Thứ tự BFS:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

Ứng dụng BFS (giải thích chi tiết)

- Tìm đường đi ngắn nhất BFS duyệt theo từng lớp nên lần đầu đến một đỉnh chính là đường đi ngắn nhất từ nguồn.
- Duyệt theo level Các đỉnh được chia thành từng lớp theo khoảng cách từ đỉnh bắt đầu.
- Kiểm tra liên thông Nếu BFS từ một đỉnh không đi được hết tất cả các đỉnh $\rightarrow$ đồ thị không liên thông.
- Giải mê cung BFS tìm được đường ra ngắn nhất trong mê cung.

Tính chất BFS (giải thích chi tiết)

- Duyệt theo lớp (level) BFS luôn xử lý các đỉnh gần trước, sau đó mới đến các đỉnh xa hơn.
- Tìm đường đi ngắn nhất Trong đồ thị không trọng số, BFS luôn cho kết quả tối ưu về số bước đi.
- Sử dụng hàng đợi (queue) Queue đảm bảo thứ tự duyệt: vào trước $\rightarrow$ ra trước (FIFO).
- Độ phức tạp: $O(V + E)$ Với V là số đỉnh, E là số cạnh.
- Tạo cây BFS Trong quá trình duyệt, BFS tạo ra một cây thể hiện đường đi ngắn nhất từ nguồn.

So sánh DFS và BFS

	DFS	**BFS**
Cấu trúc	Stack / Đệ quy	Queue
Chiến lược	Đi sâu trước	Theo lớp
Thứ tự duyệt	Một nhánh đến cuối	Lan rộng từng mức
Đường đi ngắn nhất	Không đảm bảo	Đảm bảo
Ứng dụng	Chu trình, topo	Đường ngắn nhất
Độ phức tạp	$O(V + E)$	$O(V + E)$